从头实现 makeMoE

Let's get started!

- Causal Scaled Dot Product Self Attention
- 创建 Experts
- 创建 Router
  - Top-k Gating Intuition
  - Noisy top-k Gating
- 创建 SparseMoe
- Putting it all together

> 原文：makemoe-from-scratch – Huggingface

本博客介绍了从头开始实现稀疏的专家语言模型。详细解释了模型的实施过程，包括采用稀疏混合专家取代传统的前馈神经网络，实现 top-k 门控和带噪声的 top-k 门控，以及采用 Kaiming He 初始化技术

这主要受到 Andrej Karpathy 的项目“makemore”的启发并基于该项目，并从该实现中借鉴了许多可重用的组件。就像 makemore 一样，makeMoE 也是一种自回归字符级语言模型，但使用前面提到的稀疏专家混合架构。博客的其余部分重点介绍此架构的关键元素及其实现方式。我的目标是让您在阅读此博客并逐步浏览存储库中的代码后，直观地了解这一切的工作原理。

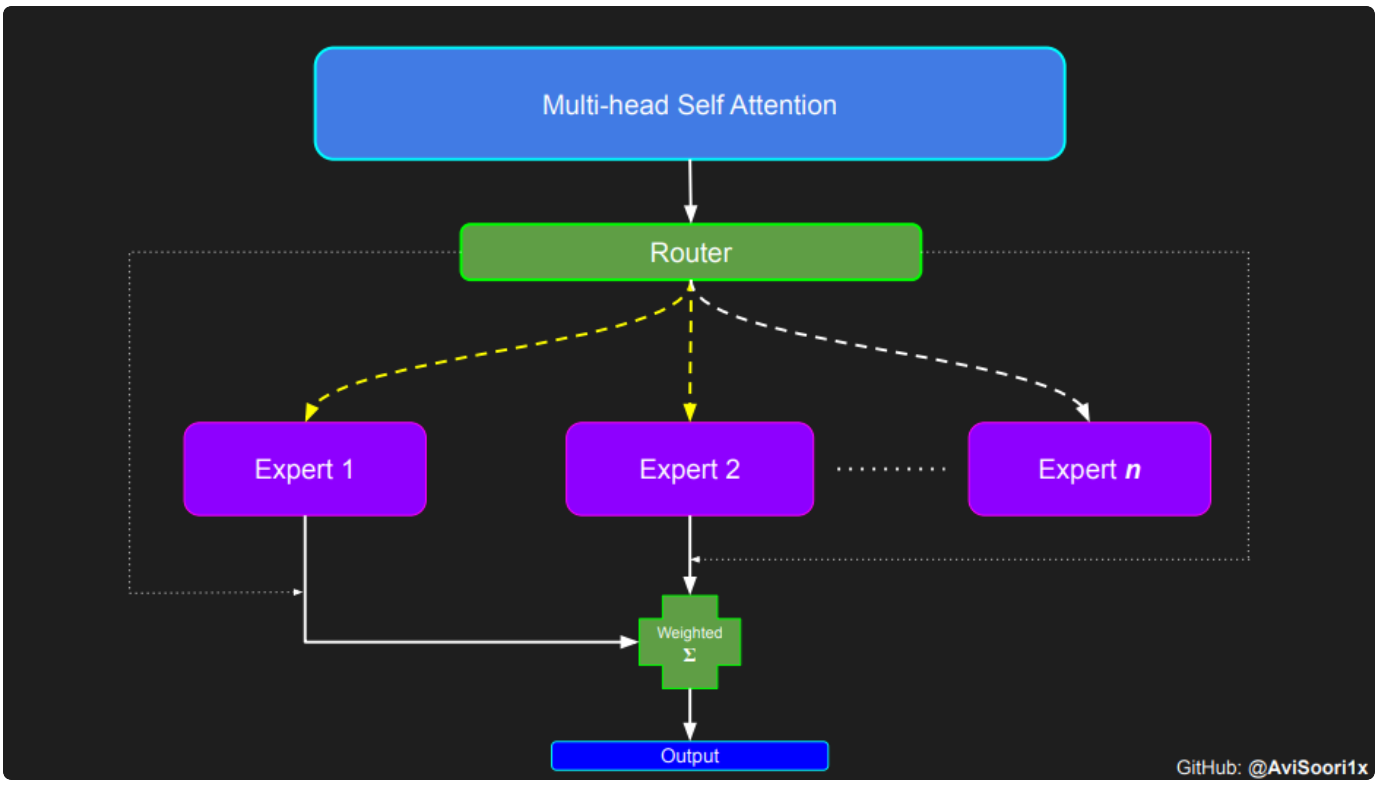
这里的 Github 存储库提供了端到端的实现：
https://github.com/AviSoori1x/makeMoE/tree/main

- makeMoE.py 是 PyTorch 的一个单文件实现。
- makMoE_from_Scratch.ipynb 文件详细讲解了整个模型架构的原理以及各个部分是如何协同工作的。我建议从这里开始。
- makeMoE_from_Scratch_with_Expert_Capacity.ipynb 只是在上述教程的基础上增加了专家容量，以实现更高效的训练。
- ~~makeMoE_Concise.ipynb 是一个可修改的集成实现，我鼓励大家对其进行修改、理解、改进并创建自己的版本。~~

With the release of Mixtral and talk of GPT-4 possibly being a mixture of experts large language model, there is significant interest in this model architecture. However, in sparse mixture of experts language models, much of the components are shared with traditional transformers. Regardless of the seeming simplicity, empirical evidence suggests that training stability is one of the main issues with these models. Hackable small scale implementations such as this may help with rapidly experimenting with new approaches.
随着 Mixtral 的发布以及 GPT-4 可能是专家大型语言模型的混合体的讨论，人们对这种模型架构产生了浓厚的兴趣。然而，在专家语言模型的稀疏混合中，许多组件与传统 Transformer 共享。尽管看起来很简单，但经验证据表明，训练稳定性是这些模型的主要问题之一。像这样可破解的小规模实现可能有助于快速试验新方法。

In this implementation I make a few significant changes from the makemore architecture:
在此实现中，我对 makemore 架构进行了一些重大更改：

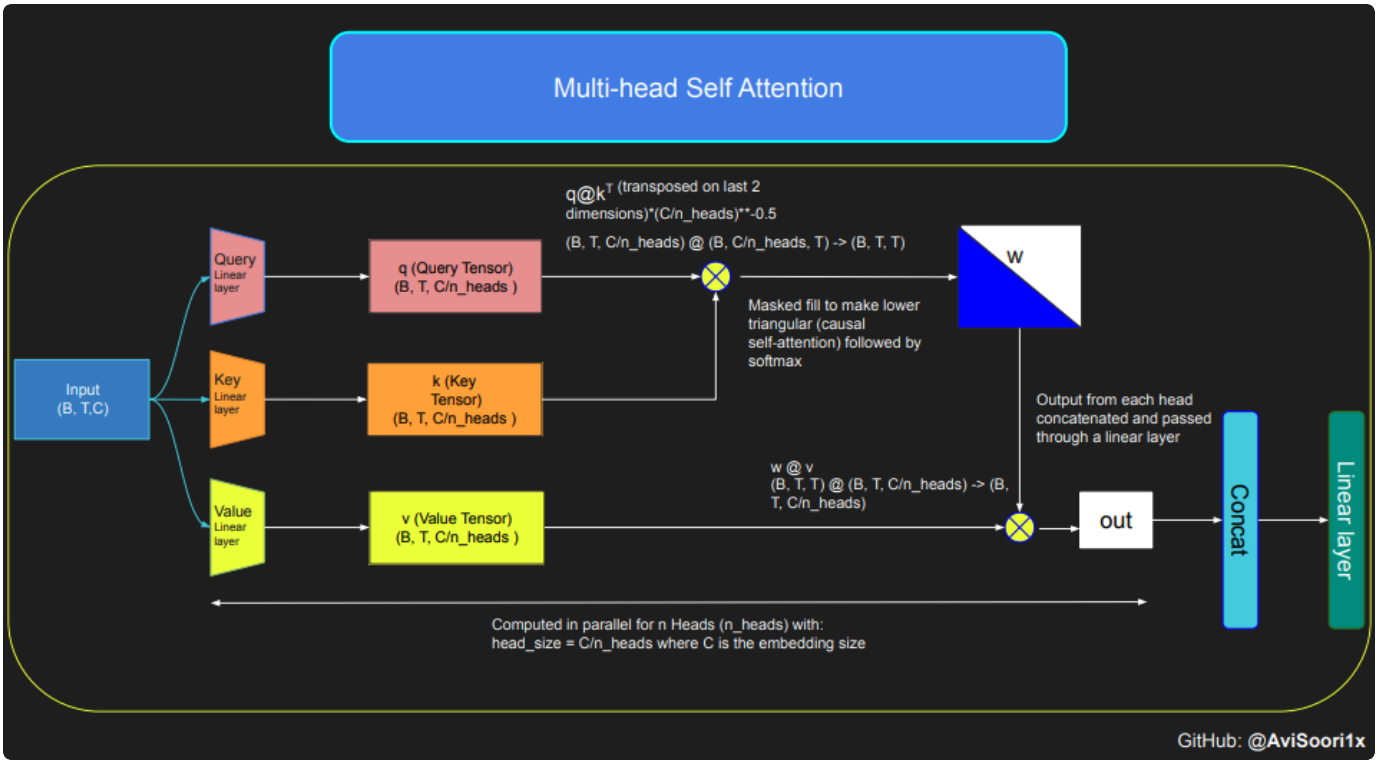
- Sparse mixture of experts instead of the solitary feed forward neural net.
  稀疏的专家混合 `sMoE`，而不是单独的前馈神经网络。
- Top-k gating and noisy top-k gating implementations.
  `top-k 门控` 和 `带噪 top-k 门控` 实现。



Let's get started!

Sparse mixture of experts language models, as anticipated, depend on self-attention for contextual comprehension. Shortly, we will explore the intricacies of the mixture of experts block. First, let's delve into self-attention to refresh our understanding.
正如预期的那样，专家语言模型的稀疏混合依赖于上下文理解的自我注意力。很快，我们将探讨专家块混合的复杂性。首先，让我们深入研究自我关注，以刷新我们的认识。

Causal Scaled Dot Product Self Attention



首先是因果自注意力机制，特别关注经典的缩放点积

```python
#This code is borrowed from Andrej Karpathy's makemore repository linked in the repo.
# The self attention layers in Sparse mixture of experts models are the same as
# in regular transformer models

torch.manual_seed(1337)
B,T,C = 4,8,32 # batch, time, channels
x = torch.randn(B,T,C)

# let's see a single Head perform self-attention
head_size = 16
key = nn.Linear(C, head_size, bias=False)
query = nn.Linear(C, head_size, bias=False)
value = nn.Linear(C, head_size, bias=False)
k = key(x)   # (B, T, 16)
q = query(x) # (B, T, 16)
wei =  q @ k.transpose(-2, -1) # (B, T, 16) @ (B, 16, T) ---> (B, T, T)

tril = torch.tril(torch.ones(T, T))
#wei = torch.zeros((T,T))
wei = wei.masked_fill(tril == 0, float('-inf'))
wei = F.softmax(wei, dim=-1) #B,T,T

v = value(x) #B,T,H
out = wei @ v # (B,T,T) @ (B,T,H) -> (B,T,H)
out.shape
```

多头因果自注意力的代码可以组织如下。

多头自注意力并行应用多个注意力头，每个注意力头都专注于通道的单独部分（嵌入维度）。多头自注意力由于固有的并行实现，从本质上改善了学习过程，提高了模型训练的效率。请注意，我在此实现过程中使用了 dropout 进行正则化，即防止过度拟合。

```python
#Causal scaled dot product self-Attention Head
n_embd = 64
n_head = 4
n_layer = 4
head_size = 16
dropout = 0.1

class Head(nn.Module):
    """ one head of self-attention """

    def __init__(self, head_size):
        super().__init__()
        self.key = nn.Linear(n_embd, head_size, bias=False)
        self.query = nn.Linear(n_embd, head_size, bias=False)
        self.value = nn.Linear(n_embd, head_size, bias=False)
        self.register_buffer('tril', torch.tril(torch.ones(block_size, block_size)))

        self.dropout = nn.Dropout(dropout)

    def forward(self, x):
        B,T,C = x.shape
        k = self.key(x)   # (B,T,C)
        q = self.query(x) # (B,T,C)
        # compute attention scores ("affinities")
        wei = q @ k.transpose(-2,-1) * C**-0.5 # (B, T, C) @ (B, C, T) -> (B, T, T)
        wei = wei.masked_fill(self.tril[:T, :T] == 0, float('-inf')) # (B, T, T)
        wei = F.softmax(wei, dim=-1) # (B, T, T)
        wei = self.dropout(wei)
        # perform the weighted aggregation of the values
        v = self.value(x) # (B,T,C)
        out = wei @ v # (B, T, T) @ (B, T, C) -> (B, T, C)
        return out
```

Multi-head self attention is implemented as follows:

多头自注意力实现如下：

```python
#Multi-Headed Self Attention
class MultiHeadAttention(nn.Module):
    """ multiple heads of self-attention in parallel """

    def __init__(self, num_heads, head_size):
        super().__init__()
        self.heads = nn.ModuleList([Head(head_size) for _ in range(num_heads)])
        self.proj = nn.Linear(n_embd, n_embd)
        self.dropout = nn.Dropout(dropout)

    def forward(self, x):
        out = torch.cat([h(x) for h in self.heads], dim=-1)
        out = self.dropout(self.proj(out))
        return out
```

创建 Experts

在稀疏混合专家（Sparse Mixture of Experts）架构中，每个 Transformer 块内的自注意力机制保持不变。然而，每个块的结构都发生了显着的变化：标准的前馈神经网络被几个稀疏激活的前馈网络（称为专家）所取代。“稀疏激活”是指序列中的每个令牌仅路由到可用池总数中有限数量的专家（通常是一两个）的过程。这有助于提高训练和推理速度，因为在每次前向传递中都会激活少数专家。然而，所有专家都必须在 GPU 内存中，因此当参数总数达到数千亿甚至数万亿时，就会产生有趣的部署问题。

```python
#Expert module
class Expert(nn.Module):
    """ An MLP is a simple linear layer followed by a non-linearity i.e. each Expert """

    def __init__(self, n_embd):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(n_embd, 4 * n_embd),
            nn.ReLU(),
            nn.Linear(4 * n_embd, n_embd),
            nn.Dropout(dropout),
        )

    def forward(self, x):
        return self.net(x)
```

创建 Router

Top-k Gating Intuition

```python
# First define the top k router module
class TopkRouter(nn.Module):
    def __init__(self, n_embed, num_experts, top_k):
        super(TopkRouter, self).__init__()
        self.top_k = top_k
        self.linear =nn.Linear(n_embed, num_experts)  # 门控网络，也称为路由器

    def forward(self, mh_ouput):
        # mh_ouput is the output tensor from multihead self attention block
        logits = self.linear(mh_output)
        top_k_logits, indices = logits.topk(self.top_k, dim=-1)  # Get top-k experts

        # 用“-inf”填充其余部分并通过 softmax 激活。
        # 这会将 '-inf' 值推到零，使前两个值更加突出，总和为 1。
        zeros = torch.full_like(logits, float('-inf'))
        sparse_logits = zeros.scatter(-1, indices, top_k_logits)

        router_output = F.softmax(sparse_logits, dim=-1)
        return router_output, indices
```

查看 top_k_logits, indices：

```
#output:
(tensor([[[ 0.0246, -0.0190],
          [ 0.1991,  0.1513],
          [ 0.9749,  0.7185],
          [ 0.4406, -0.8357]],

         [[ 0.6206, -0.0503],
          [ 0.8635,  0.3784],
          [ 0.6828,  0.5972],
          [ 0.4743,  0.3420]]], grad_fn=<TopkBackward0>),
 tensor([[[2, 3],
          [2, 1],
          [3, 1],
          [2, 1]],

         [[0, 2],
          [0, 3],
          [3, 2],
          [3, 0]]]))
```

查看sparse_logits：

```
#ouput
tensor([[[0.0000, 0.0000, 0.5109, 0.4891],
         [0.0000, 0.4881, 0.5119, 0.0000],
         [0.0000, 0.4362, 0.0000, 0.5638],
         [0.0000, 0.2182, 0.7818, 0.0000]],

        [[0.6617, 0.0000, 0.3383, 0.0000],
         [0.6190, 0.0000, 0.0000, 0.3810],
         [0.0000, 0.0000, 0.4786, 0.5214],
         [0.4670, 0.0000, 0.0000, 0.5330]]], grad_fn=<SoftmaxBackward0>)
```

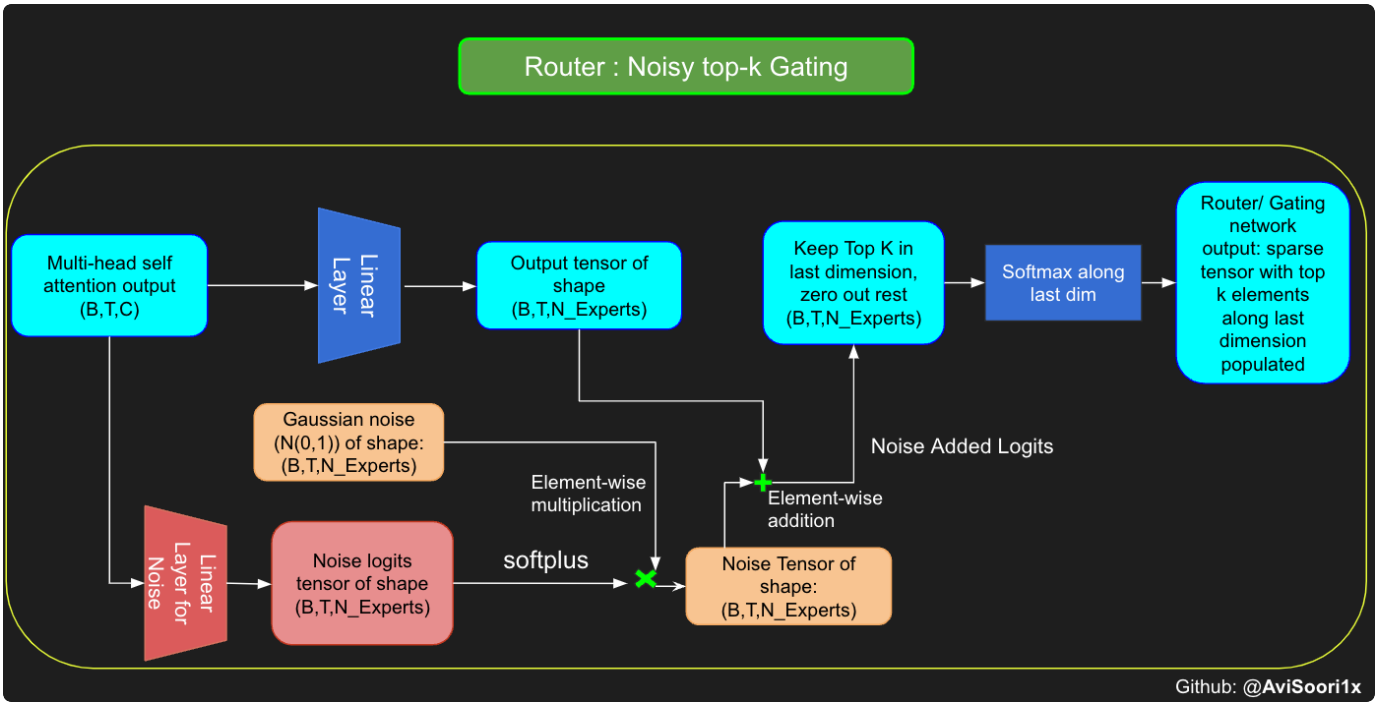
最后，让我们使用一些示例输入来测试该功能：

```python
#Testing this out:
num_experts = 4
top_k = 2
n_embd = 32

mh_output = torch.randn(2, 4, n_embd)  # Example input
top_k_gate = TopkRouter(n_embd, num_experts, top_k)
gating_output, indices = top_k_gate(mh_output)
gating_output.shape, gating_output, indices
#And it works!!
```

```plain text
#output
(torch.Size([2, 4, 4]),
 tensor([[[0.5284, 0.0000, 0.4716, 0.0000],
          [0.0000, 0.4592, 0.0000, 0.5408],
          [0.0000, 0.3529, 0.0000, 0.6471],
          [0.3948, 0.0000, 0.0000, 0.6052]],

         [[0.0000, 0.5950, 0.4050, 0.0000],
          [0.4456, 0.0000, 0.5544, 0.0000],
          [0.7208, 0.0000, 0.0000, 0.2792],
          [0.0000, 0.0000, 0.5659, 0.4341]]], grad_fn=<SoftmaxBackward0>),
 tensor([[[0, 2],
          [3, 1],
          [3, 1],
          [3, 0]],

         [[1, 2],
          [2, 0],
          [0, 3],
          [2, 3]]]))
```

Noisy top-k Gating

Noisy top-k Gating 是一种在混合专家模型（MoE）中给门控打分加入随机噪声并只选择得分最高的 k 个专家参与计算的机制，从而在保持稀疏计算的同时提升探索性和负载均衡。



Python

```python
#Changing the above to accomodate noisy top-k gating
class NoisyTopkRouter(nn.Module):
    def __init__(self, n_embed, num_experts, top_k):
        super(NoisyTopkRouter, self).__init__()
        self.top_k = top_k
        #layer for router logits
        self.topkroute_linear = nn.Linear(n_embed, num_experts)
        self.noise_linear =nn.Linear(n_embed, num_experts)

    def forward(self, mh_output):
        # mh_ouput is the output tensor from multihead self attention block
        logits = self.topkroute_linear(mh_output)

        #Noise logits
        noise_logits = self.noise_linear(mh_output)

        #Adding scaled unit gaussian noise to the logits
        noise = torch.randn_like(logits)*F.softplus(noise_logits)
        noisy_logits = logits + noise

        top_k_logits, indices = noisy_logits.topk(self.top_k, dim=-1)
        zeros = torch.full_like(noisy_logits, float('-inf'))
        sparse_logits = zeros.scatter(-1, indices, top_k_logits)
        router_output = F.softmax(sparse_logits, dim=-1)
        return router_output, indices
```

Let's test this implementation out again

让我们再次测试这个实现

```python
#Testing this out, again:
num_experts = 8
top_k = 2
n_embd = 16

mh_output = torch.randn(2, 4, n_embd)  # Example input
noisy_top_k_gate = NoisyTopkRouter(n_embd, num_experts, top_k)
gating_output, indices = noisy_top_k_gate(mh_output)
gating_output.shape, gating_output, indices
#It works!!
```

```
#output
(torch.Size([2, 4, 8]),
 tensor([[[0.4181, 0.0000, 0.5819, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000],
           [0.4693, 0.5307, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000],
           [0.0000, 0.4985, 0.5015, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000],
           [0.0000, 0.0000, 0.0000, 0.2641, 0.0000, 0.7359, 0.0000, 0.0000]],

          [[0.0000, 0.0000, 0.0000, 0.6301, 0.0000, 0.3699, 0.0000, 0.0000],
           [0.0000, 0.0000, 0.0000, 0.4766, 0.0000, 0.0000, 0.0000, 0.5234],
           [0.0000, 0.0000, 0.0000, 0.6815, 0.0000, 0.0000, 0.3185, 0.0000],
           [0.4482, 0.5518, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000]]],
         grad_fn=<SoftmaxBackward0>),
 tensor([[[2, 0],
           [1, 0],
           [2, 1],
           [5, 3]],

          [[3, 5],
           [7, 3],
           [3, 6],
           [1, 0]]]))
```

创建 SparseMoe

The Sparsely-Gated Mixture-Of-Experts layer

```python
class SparseMoE(nn.Module):
    def __init__(self, n_embed, num_experts, top_k):
        super(SparseMoE, self).__init__()
        self.router = NoisyTopkRouter(n_embed, num_experts, top_k)
        self.experts = nn.ModuleList([Expert(n_embed) for _ in range(num_experts)])
        self.top_k = top_k

    def forward(self, x):
        gating_output, indices = self.router(x)
        final_output = torch.zeros_like(x)

        # Reshape inputs for batch processing
        flat_x = x.view(-1, x.size(-1))
        flat_gating_output = gating_output.view(-1, gating_output.size(-1))

        # Process each expert in parallel
        for i, expert in enumerate(self.experts):
            # Create a mask for the inputs where the current expert is in top-k
            expert_mask = (indices == i).any(dim=-1)
            flat_mask = expert_mask.view(-1)

            if flat_mask.any():
                expert_input = flat_x[flat_mask]
                expert_output = expert(expert_input)

                # Extract and apply gating scores
                gating_scores = flat_gating_output[flat_mask, i].unsqueeze(1)
                weighted_output = expert_output * gating_scores

                # Update final output additively by indexing and adding
                final_output[expert_mask] += weighted_output.squeeze(1)

        return final_output
```

test

```python
import torch
import torch.nn as nn

#Let's test this out
num_experts = 8
top_k = 2
n_embd = 16
dropout=0.1

mh_output = torch.randn(4, 8, n_embd)  # Example multi-head attention output
sparse_moe = SparseMoE(n_embd, num_experts, top_k)
final_output = sparse_moe(mh_output)
print("Shape of the final output:", final_output.shape)
```

```plain text
Shape of the final output: torch.Size([4, 8, 16])
```

Putting it all together

多头自注意力和稀疏混合专家组合，形成稀疏混合专家变压器块（a sparse mixture of experts transformer block）。就像在普通变压器块中一样，添加了跳跃连接以确保训练稳定并避免梯度消失等问题。此外，还采用层归一化来进一步稳定学习过程。

```python
#Create a self attention + mixture of experts block, that may be repeated several number of times
class Block(nn.Module):
    """ Mixture of Experts Transformer block: communication followed by computation (multi-head self attention + SparseMoE) """

    def __init__(self, n_embed, n_head, num_experts, top_k):
        # n_embed: embedding dimension, n_head: the number of heads we'd like
        super().__init__()
        head_size = n_embed // n_head
        self.sa = MultiHeadAttention(n_head, head_size)
        self.smoe = SparseMoE(n_embed, num_experts, top_k)
        self.ln1 = nn.LayerNorm(n_embed)
        self.ln2 = nn.LayerNorm(n_embed)

    def forward(self, x):
        x = x + self.sa(self.ln1(x))
        x = x + self.smoe(self.ln2(x))
        return x
```

Finally putting it all together to crease a sparse mixture of experts language model

最后，将它们放在一起，形成稀疏的专家语言模型混合

Python

```python
class SparseMoELanguageModel(nn.Module):

    def __init__(self):
        super().__init__()
        # token embedding: 将 token id 映射为向量表示
        self.token_embedding_table = nn.Embedding(vocab_size, n_embed)
        # position embedding: 位置编码，将位置索引映射为向量
        self.position_embedding_table = nn.Embedding(block_size, n_embed)
        # Transformer Block 堆叠（包含 MoE 结构）
        self.blocks = nn.Sequential(*[
            Block(n_embed, n_head=n_head, num_experts=num_experts, top_k=top_k)
            for _ in range(n_layer)
        ])
        # 最后的 LayerNorm
        self.ln_f = nn.LayerNorm(n_embed)
        # 输出层，将隐藏状态映射到词表大小（得到 logits）
        self.lm_head = nn.Linear(n_embed, vocab_size)

    def forward(self, idx, targets=None):
        B, T = idx.shape

        # idx 和 targets 都是 (B, T) 的整数张量（token id）
        # token embedding，输出 (B, T, C)
        tok_emb = self.token_embedding_table(idx)
        # position embedding，输出 (T, C)
        pos_emb = self.position_embedding_table(torch.arange(T, device=device))
        # 相加得到输入表示 (B, T, C)
        x = tok_emb + pos_emb
        # 经过 Transformer Blocks（包含 MoE）
        x = self.blocks(x)  # (B, T, C)
        # LayerNorm
        x = self.ln_f(x)    # (B, T, C)
        # 输出 logits（每个位置对词表的预测）
        logits = self.lm_head(x)  # (B, T, vocab_size)

        if targets is None:
            loss = None
        else:
            B, T, C = logits.shape
            # reshape 成 (B*T, C)，方便计算交叉熵
            logits = logits.view(B * T, C)
            # targets reshape 成 (B*T)
            targets = targets.view(B * T)
```

```python
            # 计算语言模型的交叉熵损失
            loss = F.cross_entropy(logits, targets)

        return logits, loss

    def generate(self, idx, max_new_tokens):
        # idx: (B, T), 当前上下文 token 序列
        for _ in range(max_new_tokens):
            # 只保留最近 block_size 个 token（上下文窗口裁剪）
            idx_cond = idx[:, -block_size:]
            # 前向计算得到 logits
            logits, _ = self(idx_cond)
            # 只取最后一个时间步的预测 (B, vocab_size)
            logits = logits[:, -1, :]
            # softmax 转成概率分布
            probs = F.softmax(logits, dim=-1)
            # 按概率采样下一个 token
            idx_next = torch.multinomial(probs, num_samples=1)  # (B, 1)
            # 拼接到序列后面
            idx = torch.cat((idx, idx_next), dim=1)  # (B, T+1)

        return idx
```

初始化对于深度神经网络的高效训练非常重要。这里使用 Kaiming He initialization，这是一个可能提高模型性能的机会。

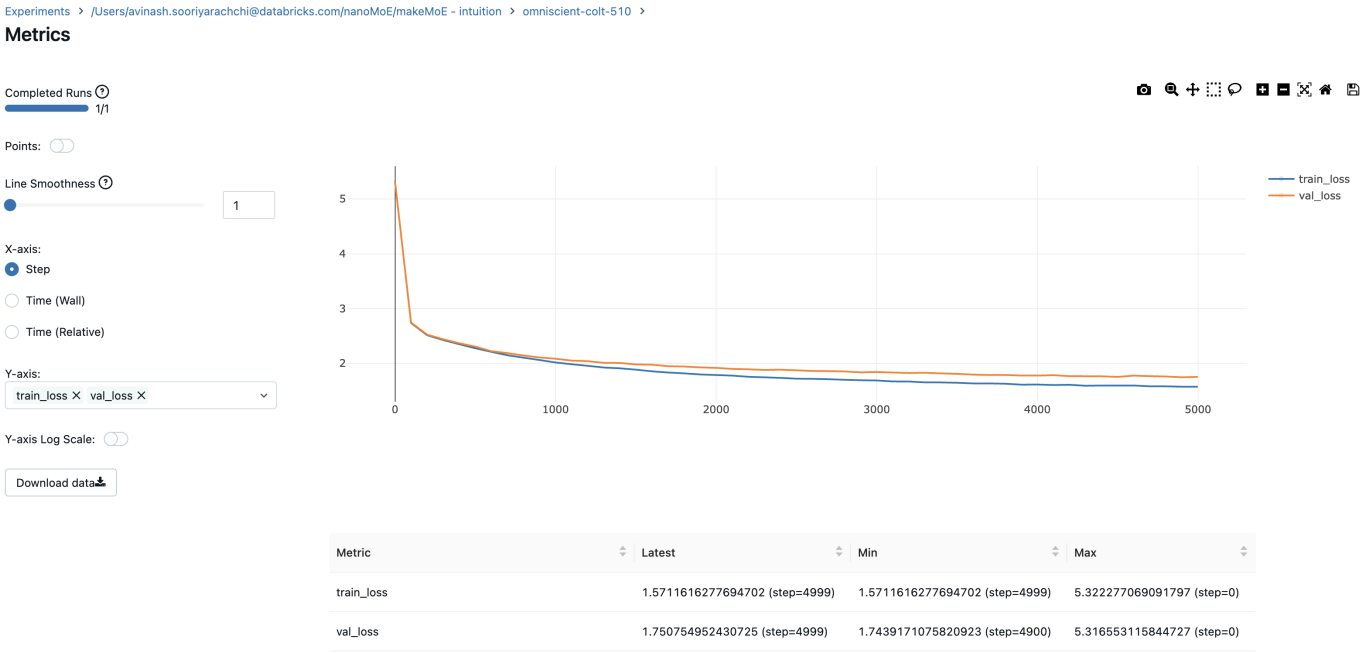
```python
def kaiming_init_weights(m):
    if isinstance (m, (nn.Linear)):
        init.kaiming_normal_(m.weight)

model = SparseMoELanguageModel()
model.apply(kaiming_init_weights)
```

使用 mlflow 来跟踪和记录重要的指标和训练超参数。我在这里展示的训练循环包含此代码。如果您更喜欢仅训练而不使用 mlflow，则 makeMoE github 存储库中的笔记本具有没有 MLFlow 的代码块。我个人发现跟踪参数和指标非常方便，尤其是在实验时。

```python
#Using MLFlow
m = model.to(device)
# print the number of parameters in the model
print(sum(p.numel() for p in m.parameters())/1e6, 'M parameters')

# create a PyTorch optimizer
optimizer = torch.optim.AdamW(model.parameters(), lr=learning_rate)
#mlflow.set_experiment("makeMoE")
with mlflow.start_run():
    #If you use mlflow.autolog() this will be automatically logged. I chose to explicitly log here for completeness
    params = {"batch_size": batch_size , "block_size" : block_size, "max_iters": max_iters, "eval_interval": eval_interval,
              "learning_rate": learning_rate, "device": device, "eval_iters": eval_iters, "dropout" : dropout, "num_experts": num_experts, "top_k": top_k }
    mlflow.log_params(params)
    for iter in range(max_iters):

        # every once in a while evaluate the loss on train and val sets
        if iter % eval_interval == 0 or iter == max_iters - 1:
            losses = estimate_loss()
            print(f"step {iter}: train loss {losses['train']:.4f}, val loss {losses['val']:.4f}")
            metrics = {"train_loss": losses['train'], "val_loss": losses['val']}
            mlflow.log_metrics(metrics, step=iter)


        # sample a batch of data
        xb, yb = get_batch('train')

        # evaluate the loss
        logits, loss = model(xb, yb)
        optimizer.zero_grad(set_to_none=True)
        loss.backward()
        optimizer.step()
```

```
8.996545 M parameters
step 0: train loss 5.3223, val loss 5.3166
step 100: train loss 2.7351, val loss 2.7429
step 200: train loss 2.5125, val loss 2.5233
.
.
.

step 4999: train loss 1.5712, val loss 1.7508
```

Logging train and validation losses gives you a good indication of how the training is going. The plot shows that I probably should have stopped around 4500 steps (when the validation loss slightly jumps up)
记录训练和验证损失可以很好地指示训练的进展情况。该图显示，我可能应该在 4500 步左右停止（当验证损失略有上升时）



Now we can generate text using this model character by character, autoregressively. For a sparsely activated ~9M parameter model, I can't complain.
现在我们可以使用这个模型逐个字符自回归地生成文本。对于稀疏激活的 ~9M 参数模型，我不能抱怨。

Python

```python
# generate from the model. Not great. Not too bad either
context = torch.zeros((1, 1), dtype=torch.long, device=device)
print(decode(m.generate(context, max_new_tokens=2000)[0].tolist()))
```

Plain Text

```
DUKE VINCENVENTIO:
If it ever fecond he town sue kigh now,
That thou wold'st is steen 't.

SIMNA:
Angent her; no, my a born Yorthort,
Romeoos soun and lawf to your sawe with ch a woft ttastly defy,
To declay the soul art; and meart smad.

CORPIOLLANUS:
Which I cannot shall do from by born und ot cold warrike,
What king we best anone wrave's going of heard and good
Thus playvage; you have wold the grace.
...
```